

INFO-I 211 - Information Infrastructure II

Practice Quiz - Python and Programming

Solutions + Grading

Do not open until told to do so.

Answer questions in the space provided. Draw a box around your final answer if necessary.

Name: Alexander Hayes - Notes

IU Email: hayesall@iu.edu

Which section do you attend?

- ☒ Alexander Hayes - M/W - 9:35–10:50 AM
- ☐ Erika Lee - M/W - 11:10–12:25 AM
- ☐ Alexander Hayes - Tu/Th - 3:55–5:10 PM

General Instructions:

1. You have **60 minutes** to complete this exam. If you must leave this room: you must submit your exam.
2. Using any electronic device (watches, phones, earbuds, etc.) will be met with consequences. Put them away.
3. Include all syntax. e.g. If a program prints a list of strings, its output should look like: ['a', 'b', 'c'].
4. The **maximum score** is 20 points, including bonus.
5. Questions are arranged roughly by difficulty, with easier ones at the beginning and harder ones at the end. The following table breaks down how you might spend your time:

Question	Lower Question Time	Upper Question Time	Lower Clock Time	Upper Clock Time
Q1	7 min.	10 min.	07:00	10:00
Q2	11 min.	15 min.	18:00	25:00
Q3	14 min.	20 min.	32:00	45:00
Q4	10 min.	15 min.	42:00	60:00
Q5 (Bonus)	18 min.	Skip	60:00	Skip

Question:	1	2	3	4	5	Total
Points:	5	6	4	5	0	20
Bonus Points:	0	0	0	0	3	3
Score:						

we try to add a table like this to give a sense for how long a question might take.

Answers in blue

Grading notes ↳ orange

1. Answer questions about the following Python program:

```
1 x = 5
2 y = 3
3 print((x * y) + 1)
4 y = 2
5 print((x * y) + 1)
```

(a) [1 pt.] What is the **data type** of each variable?

x: **int**

y: **int**

+0.5 : per correct answer

0 : anything else

(b) [1 pt.] Line 3 has an expression inside a print. What values are x and y bound to when line 3 is evaluated?

x = **5**

y = **3**

(c) [1 pt.] Line 5 has an expression inside a print. What values are x and y bound to when line 5 is evaluated?

x = **5**

y = **2**

(d) [2 pts.] What final result is shown when this program runs?

16
11

2 : solution, 2 lines

1 : right idea, 1 line

16 11

0.5: partial reasoning,
like

16
16

2. Answer questions about the following program.

```
1 def foo(x: int) -> int:
2     return x * 2
3
4 def bar(x: int) -> int:
5     if x % 2 == 0:
6         return (x * 3) + 2
7     return (x * 3) + 1
```

+1: "No" answer

+1: Some reasoning, look for "call", "print", "evaluation"

(a) [2 pts.] As written, does running this program show/output anything? Why or why not?

No. Functions are defined, but are not called or evaluated in the program. No print calls.

(b) [4 pts.] Assume the functions `foo(x)` and `bar(x)` take integer inputs and return integer outputs. Fill in the following table, starting from a function input `x`, and showing what is returned (1 pt. per row).

Input x	foo(x)	bar(x)	bar(foo(x))
0	0	2	2
1	2	4	8
2	4	8	14
3	6	10	20

} Full credit:
+4 for filling
in the table
like this.

+0.5 if 2/3 values in a row
are correct, so:

Input x	foo(x)	bar(x)	bar(foo(x))
0	0	2	0
	✓	✓	✗

... row would receive 0.5

```

1 def foo(x: int) -> int:
2     return x * 2
3
4 def bar(x: int) -> int:
5     if x % 2 == 0:
6         return (x * 3) + 2
7     return (x * 3) + 1

```

Tracing how
these functions
work:

foo → doubles its input x

`>>> foo(0)`
0

`>>> foo(1)`
1

bar → multiplies by 3 and adds
2/1 depending on whether
 x is even/odd ($x \% 2 == 0$)

`>>> bar(0)`

$0 \% 2 == 0$ ✓
⇒ $(0 * 3) + 2$

2

`>>> bar(1)`

$1 \% 2 == 0$ ✗
⇒ $(1 * 3) + 1$

4

bar(foo(x)) → double, then add something.

Since **foo** always returns
an even number, we'll always
return $(x * 3) + 2$

`>>> foo(2)`

4

`>>> bar(foo(2))`
 $(4 * 3) + 2$

14

3. Consider the following Python program:

```
1  # Initialize variables for the program
2  i: int = 0
3  v: int = 1
4
5  # Compute a result
6  while (i < 4):
7      if (i % 2 == 0):
8          v = v * 2
9          i = i + 1
10
11 # Handle showing outputs
12 while (i != 0):
13     print(i, v, i * v)
14     i = i - 1
15     v = v - 1
```

- (a) [1/2 pt.] How many **expressions** are in the print statement on line 13? (*Reminder*: a variable by itself is also an expression. You can also think of this question as asking: “how many things are printed?”)

3

+ 0.5 : correct

0 : otherwise

$\text{print}(i, v, i * v)$
 ↓ ↓ ↓
 1 2 3
∴ 3 things printed per line

- (b) [1/2 pt.] How many times does the if statement on line 7 evaluate to True?

2

- (c) [1 pt.] What does the variable “v” contain between lines 10-11?

4

+ 1 : correct

0 : otherwise

- (d) [2 pts.] What final output gets shown after running this program?

4	4	16
3	3	9
2	2	4
1	1	1

} + 0.5 per correct “row”
in the right order
(spacing is not
important, but
should be 4 x 3,
one line each)

```

1  # Initialize variables for the program
2  i: int = 0
3  v: int = 1
4
5  # Compute a result
6  while (i < 4):
7      if (i % 2 == 0):
8          v = v * 2
9          i = i + 1
10
11 # Handle showing outputs
12 while (i != 0):
13     print(i, v, i * v)
14     i = i - 1
15     v = v - 1

```

Tracing programs
with conditions
and loops.

Lines 2-3: think of i & v like boxes with
a number inside

i	0
v	1

Lines 6-9: loop while ($i < 4$)

first iteration | while ($i < 4$):
 if ($i \% 2 == 0$): True
 $v = v * 2 \Rightarrow v = 2$
 $i = i + 1 \Rightarrow i = 1$

(b) updates 2 times

i	v
0	2
1	-
2	4
3	-
4	-

When the loop breaks:

$i = 4$, $v = 4$ → (c)

Lines 12-15 : breaks when i reaches 0,
decrements i and v by 1

4. [5 pts.] There are several errors in the following program. It is supposed to:

1. Get 6 inputs from the user (using `input()`)
2. Convert them from strings to integers and append them to a list
3. Iterate over the list, printing numbers that are both divisible by 2 and less than or equal to 10

But there are several errors. **Circle and correct 5 errors** (there may be more). Errors may be syntactic or semantic. (Hint: Some errors might be easier to identify by tracing the program.)

```
1  # Declare variables
2  A: list[int] = []
3  i: int = 0
4
5  # Read input from the user
6  i = 0
7  while (i < 6):
8      A[i] = int(input("Choose a number> "))
9      i = i + 1
10
11 # Process data and output results
12 while (i < 5):
13     if (A[i] % 2 == 1) or (A[i] < 10):
14         print(A[i])
15     else:
16         print(A[i])
17     i = i + 1
```

Handwritten annotations and corrections:

- Line 7: `while (i < 6):` → Should be `A.append(...)`
- Line 8: `A[i] = int(input("Choose a number> "))` → Indent right to be in the loop.
- Line 9: `i = i + 1` → Indent right to be in the loop.
- Line 10: `i = 0` → Reset `i = 0`
- Line 12: `while (i < 5):` → `6`
- Line 13: `if (A[i] % 2 == 1) or (A[i] < 10):` → `and`
- Line 13: `== 1` → Should be `== 0` or `!= 1` to represent "divisible by 2"
- Line 14: `print(A[i])` → Should print `A[i]`
- Line 15: `else:` → Remove

+0.5 per error circled, +0.5 for right explanation

(Try to err in their favor if an explanation is pretty close.)

0 points if something is circled but has no explanation

5. (Bonus) We want to simplify this program:

```

1  v1 = 2
2  v2 = 3
3  v3 = 2
4  v4 = 0
5
6  while (v4 < 4):
7      v1 = v1 + v1
8      v4 = v4 + 1
9
10 while (v3 < v4):
11     print(v1 - v3)
12     v3 = v3 + 1

```

At this point:

$v1 = 32$
 $v2 = 3$ (but is not used)
 $v3 = 2$
 $v4 = 4$

first iteration

$while (v3 < v4):$
 $print(v1 - v3)$
 $v3 = v3 + 1$

Output looks like:

30
29

(a) [1 point (bonus)] Which variable(s) are initialized/created, but never used/updated?

$v2$

+1 : correct

0 : incorrect

(b) [1 point (bonus)] What **values** are in $v4$ and $v1$ when the first **while** loop (lines 6-8) finishes (e.g. at line 9)? You **do not** need to fill in the table, but it may help track variable state before making your final answer.

$v4: 4$

$v1: 32$

+0.5 each

0 : incorrect for both

(don't worry about the table when grading)

Loop iteration	$v4$	$v1$
Before the loop	0	2
$0 < 4$	1	4
$1 < 4$	2	8
$2 < 4$	3	16
$3 < 4$	4	32
Break on $4 < 4$	4	32

(c) [1 point (bonus)] An equivalent program can be written which uses only one variable. Complete the following program's $v1$ variable (line 1), and **while** loop condition (line 3). (Hint: What does the original program print?)

```

1  v1 = 30
2
3  while ( v1 > 28 ):
4      print(v1)
5      v1 = v1 - 1

```

or: $v1 \geq 29$

+0.5 x 2 answers

0 : otherwise

Computing the final grade:

Question:	1	2	3	4	5	Total
Points:	5	6	4	5	0	20
Bonus Points:	0	0	0	0	3	3
Score:	5	6	4	5	3	20

Maximum grade
is 20, bonus points
can "catch up" to 20.

e.g. All correct
and all bonus

i.e. :

>>> min(23, 20)
20

Some
correct

>>> min(18.5, 20)
18.5